

PRACTICAL 27 -

Objective - Objective - Write a program to represent two polynomials using singly linked lists and perform addition of these two polynomials.

CODE -

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Definition of a node in the linked list
5  typedef struct PolynomialNode {
6      int coefficient;
7      int exponent;
8      struct PolynomialNode* next;
9  } PolynomialNode;
10
11 // Function to create a new node
12 PolynomialNode* create_node(int coefficient, int exponent) {
13     PolynomialNode* new_node = (PolynomialNode*)malloc(sizeof(
14         PolynomialNode));
15     new_node->coefficient = coefficient;
16     new_node->exponent = exponent;
17     new_node->next = NULL;
18     return new_node;
19 }
20 // Function to insert a node into the polynomial in sorted order (by
21 // exponent)
22 PolynomialNode* insert_node(PolynomialNode* head, int coefficient, int
23     exponent) {
24     PolynomialNode* new_node = create_node(coefficient, exponent);
25     if (!head || head->exponent < exponent) {
26         new_node->next = head;
27         return new_node;
28     }
29     PolynomialNode* temp = head;
30     while (temp->next && temp->next->exponent >= exponent) {
31         temp = temp->next;
32     }
33     if (temp->exponent == exponent) {
34         temp->coefficient += coefficient; // Combine terms with same
35         exponent
36         free(new_node);
37     } else {
38         new_node->next = temp->next;
39         temp->next = new_node;
40     }
41 }
```

```
38     return head;
39 }
40
41 // Function to display a polynomial
42 void display_polynomial(PolynomialNode* head) {
43     PolynomialNode* temp = head;
44     while (temp) {
45         printf("%dx^%d", temp->coefficient, temp->exponent);
46         if (temp->next) printf(" + ");
47         temp = temp->next;
48     }
49     printf("\n");
50 }
51
52 // Function to add two polynomials
53 PolynomialNode* add_polynomials(PolynomialNode* poly1, PolynomialNode*
poly2) {
54     PolynomialNode* result = NULL;
55     while (poly1 && poly2) {
56         if (poly1->exponent > poly2->exponent) {
57             result = insert_node(result, poly1->coefficient, poly1->
exponent);
58             poly1 = poly1->next;
59         } else if (poly1->exponent < poly2->exponent) {
60             result = insert_node(result, poly2->coefficient, poly2->
exponent);
61             poly2 = poly2->next;
62         } else {
63             result = insert_node(result, poly1->coefficient + poly2->
coefficient, poly1->exponent);
64             poly1 = poly1->next;
65             poly2 = poly2->next;
66         }
67     }
68     while (poly1) {
69         result = insert_node(result, poly1->coefficient, poly1->
exponent);
70         poly1 = poly1->next;
71     }
72     while (poly2) {
73         result = insert_node(result, poly2->coefficient, poly2->
exponent);
74         poly2 = poly2->next;
75     }
76     return result;
77 }
78
79 // Main function to demonstrate polynomial addition
80 int main() {
81     PolynomialNode* poly1 = NULL;
82     PolynomialNode* poly2 = NULL;
```

```
83
84     // Create first polynomial:  $3x^3 + 4x^2 + 2$ 
85     poly1 = insert_node(poly1, 3, 3);
86     poly1 = insert_node(poly1, 4, 2);
87     poly1 = insert_node(poly1, 2, 0);
88
89     // Create second polynomial:  $5x^2 + 6x + 1$ 
90     poly2 = insert_node(poly2, 5, 2);
91     poly2 = insert_node(poly2, 6, 1);
92     poly2 = insert_node(poly2, 1, 0);
93
94     printf("First Polynomial: ");
95     display_polynomial(poly1);
96
97     printf("Second Polynomial: ");
98     display_polynomial(poly2);
99
100    // Add the two polynomials
101    PolynomialNode* result = add_polynomials(poly1, poly2);
102
103    printf("Resultant Polynomial after Addition: ");
104    display_polynomial(result);
105
106    return 0;
107 }
```

OUTPUT -

```
First Polynomial:  $3x^3 + 4x^2 + 2x^0$ 
Second Polynomial:  $5x^2 + 6x^1 + 1x^0$ 
Resultant Polynomial after Addition:  $3x^3 + 9x^2 + 6x^1 + 3x^0$ 
```